



Polytech Clermont

Ingénierie Mathématique et Data Science, 4^e année

Rapport de projet : Conception d'un jeu de Tetris

Présenté par : Etienne ESCOFIER, Andrea GELIE, Adrien
LABAYLE, Thomas MAS, Camille RIVIERE, Aurélien SAIS

Année 2025 / 2026

Ce rapport présente la conception et le développement d'un jeu vidéo Tetris réalisé dans le cadre d'un projet de génie logiciel et de programmation orientée objet. L'objectif principal de celui-ci était d'implémenter une version fonctionnelle du jeu en utilisant le langage C++ et la bibliothèque graphique Qt, tout en appliquant les principes de modélisation UML.

Ce projet a suivi une méthodologie structurée comprenant la planification des tâches à l'aide d'un diagramme de Gantt, la modélisation du système par différents diagrammes UML, puis l'implémentation du jeu. Une attention particulière a été portée à la conception orientée objet, notamment à travers la définition des classes principales et à l'implémentation de fonctionnalités indispensables comme la gestion des collisions, la suppression des lignes complètes et le calcul du score.

Le résultat final est donc un jeu de Tetris doté d'une interface graphique, de plusieurs niveaux de difficulté, d'un système de score évolutif ainsi que d'une fonctionnalité de sauvegarde et de reprise de partie.

Mots clés : Tetris ; Programmation orientée objet ; UML ; C++ ; Qt

This report presents the design and development of a Tetris video game made for a software engineering and object-oriented programming project. The main objective was to implement a functional version of the game using the C++ programming language and the Qt graphical library, while applying UML modeling principles.

The project followed a structured methodology, including task planning using a Gantt chart, system modeling through various UML diagrams, and finally the implementation of the game. Particular attention was paid to object-oriented design, especially through the definition of the main classes and the implementation of essential features such as collision management, full line removal, and score calculation.

The final result is a Tetris game featuring a graphical user interface, multiple difficulty levels, a progressive scoring system, as well as a save and resume functionality.

Keywords: Tetris; Object-oriented programming; UML; C++; Qt

TABLE DES MATIERES

<u>1</u>	<u>INTRODUCTION ET PRESENTATION DU SUJET</u>	<u>4</u>
<u>2</u>	<u>PRESENTATION DU JEU</u>	<u>5</u>
<u>3</u>	<u>PLANIFICATION DU TRAVAIL</u>	<u>6</u>
<u>4</u>	<u>PRESENTATION DES OUTILS UTILISES</u>	<u>7</u>
<u>5</u>	<u>DIAGRAMMES UML ET POINTS IMPORTANTS DU CODE</u>	<u>8</u>
5.1	DIAGRAMMES UML	8
5.2	POINTS IMPORTANTS DU CODE	17
<u>6</u>	<u>PRESENTATION DU JEU OBTENU</u>	<u>19</u>
<u>7</u>	<u>BILAN PERSONNEL</u>	<u>22</u>
	<u>CONCLUSION</u>	<u>23</u>
	<u>BIBLIOGRAPHIE</u>	<u>24</u>

1 INTRODUCTION ET PRESENTATION DU SUJET

Dans le cadre de notre cours de génie logiciel et programmation orientée objet, il nous a été demandé de réaliser un projet ayant pour objectif final de réaliser sur ordinateur le jeu vidéo Tetris.

Pour cela, il nous a été demandé d'appliquer les différents enseignements que nous avons reçu durant ce cours et de grandement se baser sur la programmation orientée objet pour réaliser le jeu.

Nous avons également dû faire en sorte d'utiliser le langage UML pour modéliser l'application ainsi que le langage C++ couplé à la bibliothèque graphique Qt pour la développer et faire fonctionner notre jeu de Tetris.

De plus, dans le cadre de ce projet plusieurs documents complémentaires ont été demandés afin d'accompagner le développement du jeu et d'en assurer la compréhension que ce soit pour l'utilisateur ou pour un développeur amené à reprendre notre application.

Tout d'abord, nous avons rédigé un manuel utilisateur ayant pour objectif de présenter notre jeu à un utilisateur afin qu'il comprenne les différentes fonctionnalités du jeu ainsi que son mode de fonctionnement. Ce document explique les règles du jeu, les différentes actions possibles et l'utilisation.

Ensuite, nous sommes passé au manuel développeur qui, comme son nom l'indique, s'adresse à un développeur qui devrait reprendre notre logiciel.

Enfin, nous avons réalisé un rapport de projet qui nous a permis de faire synthèse du travail que nous avons réalisé.

2 PRESENTATION DU JEU

Le Tetris[1] est un jeu vidéo de puzzle en mouvement originalement créé par l'ingénieur Alekseï Pajitnov et dont la première version est sortie en 1984 et ayant la particularité de ne jamais se terminer par la victoire du joueur ayant initié la partie (voir Figure 1).

Le principe de ce jeu est de permettre au joueur de réaliser des lignes complètes au sein de la grille à l'aide de blocs possédant diverses formes et appelés tétrominos. En règle générale, il en existe 7 formes disponibles dans le jeu et ils sont colorés de manière aléatoire. Ainsi, ceux-ci défilent du haut vers le bas de la grille et pendant ce laps de temps, le joueur a l'occasion de les déplacer dans le but de compléter le plus de lignes pour pouvoir libérer l'espace de la grille et obtenir le maximum de points.



Figure 1 : Exemple d'une fin de partie sur le clone libre du jeu JsTetris[1]

Ces déplacements peuvent être uniquement des déplacements latéraux ou des rotations et ne sont plus possibles dès lors que le bloc touche une pièce déjà placée auparavant sur la grille. De plus, même si les déplacements accessibles au joueur semblent limités, il peut tout de même avoir la possibilité d'accélérer la chute des blocs lors de leur descente.

Le score quant à lui, est généralement calculé de la manière suivante : compléter une seule ligne rapporte 40 points, alors qu'en compléter 2 en rapporte 100, 3 en rapporte 300 et 4 en rapporte 1200.

Enfin, plus il y a de lignes complétées, plus son niveau augmente et par conséquent la valeur en points d'une ligne complétée ainsi que la vitesse de chute des blocs augmentent elles aussi. L'augmentation de point lorsque le niveau augmente ce fait notamment selon l'équation $F(p,n) = p(n+1)$ où p représente le nombre de points au niveau 0 et n le niveau.

3 PLANIFICATION DU TRAVAIL

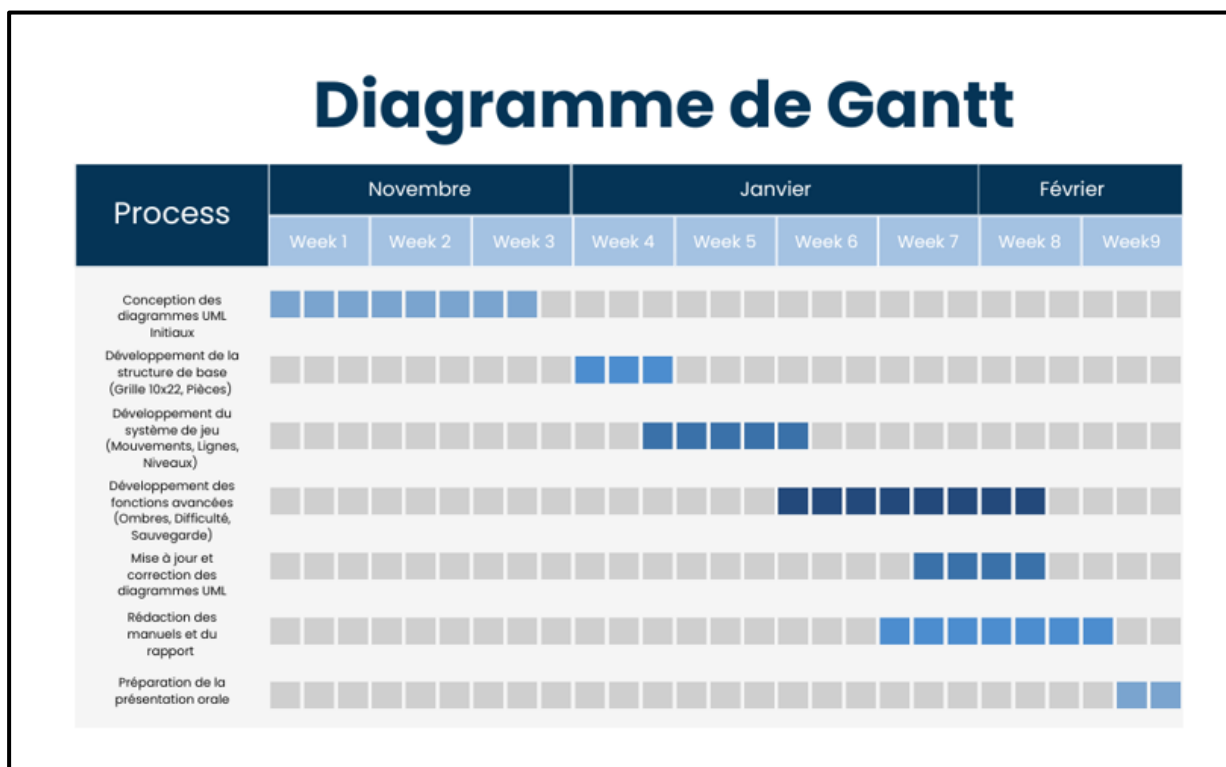


Figure 2 : Diagramme de Gantt

Dans le but de planifier plus efficacement les différentes phases de notre projet, de la conception des diagrammes UML jusqu'à la livraison de celui-ci, nous avons établi un diagramme de Gantt (voir Figure 2).

Il met notamment en évidence les différentes phases de notre projet. Ainsi, la phase de développement du jeu, divisée en trois parties, est celle qui est la plus longue. En effet, on retrouve l'étape de création de la structure de base de jeu avant de pouvoir passer au système en lui-même puis aux fonctionnalités avancées. Cette dernière partie chevauche deux autres phases de notre projet. Puisque cela correspond au moment où le codage est presque terminé, c'est naturellement la période choisie pour pouvoir mettre à jour nos diagrammes UML ainsi que pour rédiger les différents manuels et rapports demandés.

4 PRESENTATION DES OUTILS UTILISES

Lors de la réalisation de ce projet, nous avons utilisé de nombreux outils tels que Visual Paradigm[2] and Lucidchart[3] qui sont les deux plateformes avec lesquelles nous avons créé les diagrammes UML qui seront présenté dans la partie 5 de ce rapport. Ces outils ont donc une part assez importante dans ce projet puisque grâce aux diagrammes qu'ils nous ont permis de réaliser, nous avons pu commencer à coder notre jeu en ayant une vision bien plus claire de ce que nous souhaitions faire.

Nous avons également utilisé Qt Creator[4] qui est une bibliothèque graphique orientée objet codée en C++. Elle a été l'outil central de notre projet puisque c'est sur celle-ci que nous avons développé et codé le code permettant à notre jeu de Tetris de fonctionner et c'est également grâce à elle que nous pouvons avoir une interface graphique plus avancée pour notre jeu.

Afin de rendre l'esthétique de notre jeu plus agréable, nous avons utilisé l'outil Convertio [5] pour convertir et adapter les images nécessaires pour la création et l'adaptation du fond graphique du jeu.

Enfin, pour l'intégration de la musique au jeu, nous avons utilisé noTube [6]. Cet outil nous a permis de récupérer des fichiers audios destinés à être intégrés dans celui-ci.

5 DIAGRAMMES UML ET POINTS IMPORTANTS DU CODE

5.1 DIAGRAMMES UML

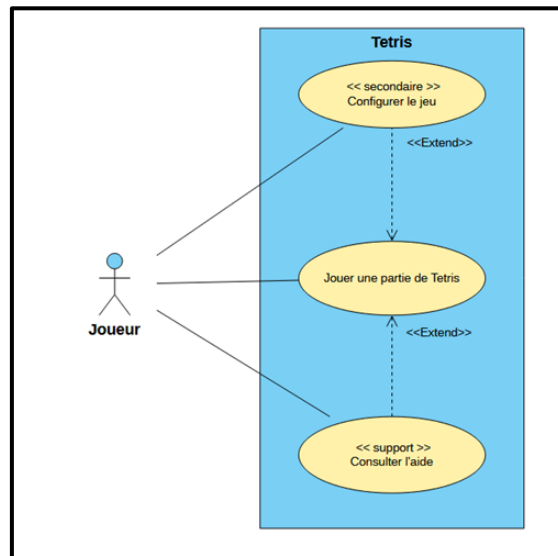


Figure 3 : Diagramme des cas d'utilisation

Le diagramme de cas d'utilisation ci-dessus (voir Figure 3) représente les principales interactions entre l'utilisateur et le jeu de Tetris que nous avons créé. Il permet notamment de décrire, de manière globale, les possibilités offertes au joueur, qui est l'acteur principal, lors de l'utilisation du jeu. Ici, le cas d'utilisation principal est « Jouer une partie de Tetris », qui correspond à l'objectif central du logiciel. De plus, il existe également deux cas d'utilisation secondaires : « Configurer le jeu », qui permet au joueur d'ajuster certains paramètres (la difficulté) et « Consulter l'aide », qui permet au joueur d'accéder à des informations expliquant les règles du jeu. Ce diagramme de cas d'utilisation a constitué la base initiale de la conception du jeu. Il a permis de définir les fonctionnalités essentielles à implémenter et de structurer les premières phases de développement. Au fur et à mesure de l'avancement du projet, le jeu a toutefois été enrichi par l'ajout de fonctionnalités supplémentaires non représentées dans ce diagramme initial, comme la possibilité de sauvegarder et de reprendre une partie.

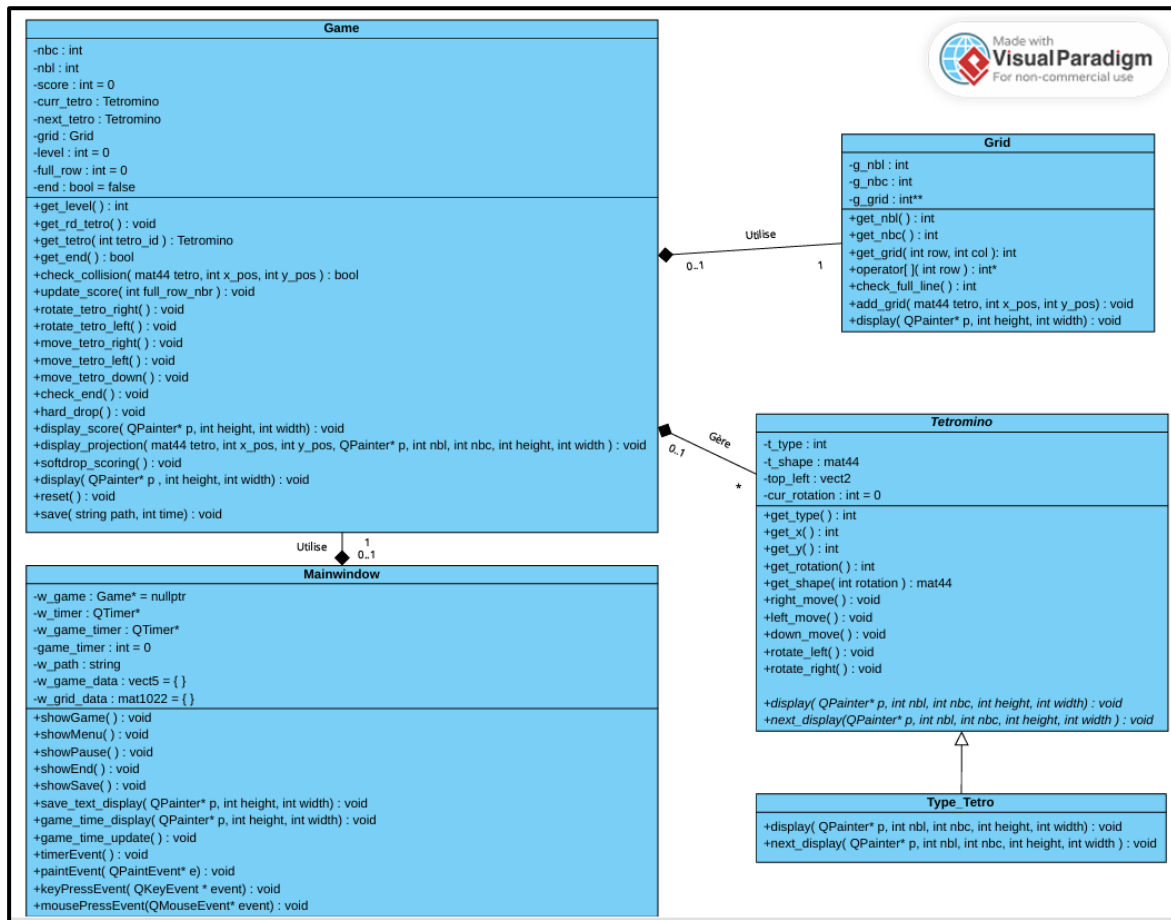


Figure 4 : Diagramme de classes

Ce diagramme de classes UML (voir Figure 4) modélise la structure du jeu en décrivant les principales classes qui sont Game, Grid, Tetromino, Type_Tetro (Une pour chaque forme de tétramino disponible) et Mainwindow, leurs attributs, leurs méthodes ainsi que les relations entre elles.

On retrouve également dans ce diagramme les notation mat44, mat1022 et vect5 qui correspondent respectivement à une matrice de taille 4x4, une de taille 10x22 et un vecteur de taille 5.

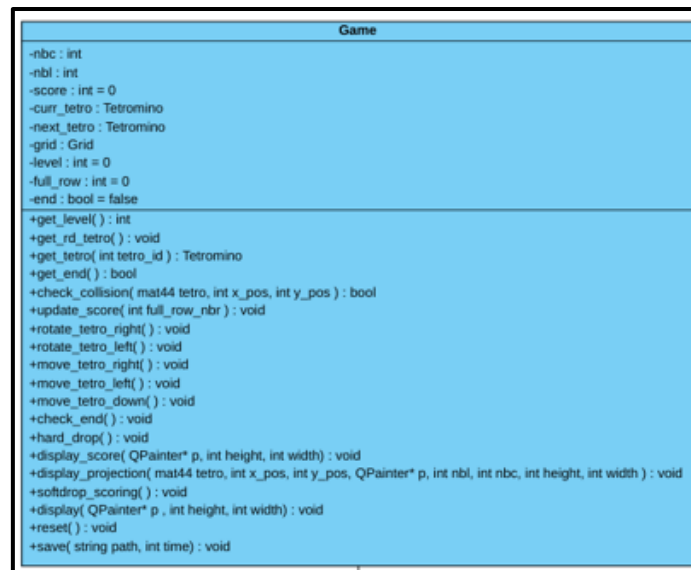


Figure 5 : Classe Game

Ici (voir Figure 5), on retrouve la classe qui est au centre du jeu, la classe Game qui correspond à la partie. Elle fonctionne notamment en utilisant la classe Grid (voir Figure 6) et en gérant les tétraminoes (voir Figure 7).

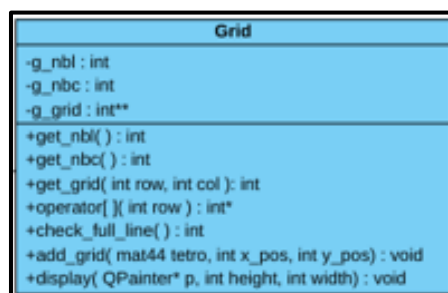


Figure 6 : Classe Grid

La classe Grid (voir Figure 6) va nous permettre de gérer tout ce qui concerne le puit lors de la partie et va donc permettre à celle-ci d'avoir accès aux méthodes lui permettant de vérifier l'état du puit pour pouvoir notamment mettre le score à jour.

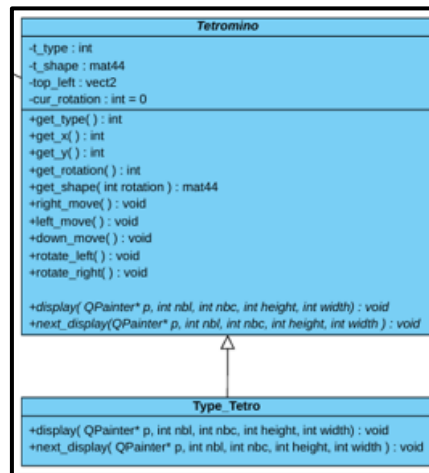


Figure 7 : Classe Tetromino et héritage

Le morceau de diagramme ci-dessus (voir Figure 7) correspond à la classe abstraite Tetromino qui va notamment nous servir pour implémenter les méthodes en relation avec la gestion des blocs et leur affichage pendant la partie. De plus, nous avons mis en place la relation d'héritage entre celle-ci et la sous-classe Type_Tetro qui va nous permettre de définir les méthodes display et next-display qui permettent respectivement d'afficher le tétramino courant et le prochain tétramino. Enfin, il existe en réalité sept classes Type_Tetro dans notre code puisque chacune d'entre-elles nous permet de gérer une forme précise de tétramino.

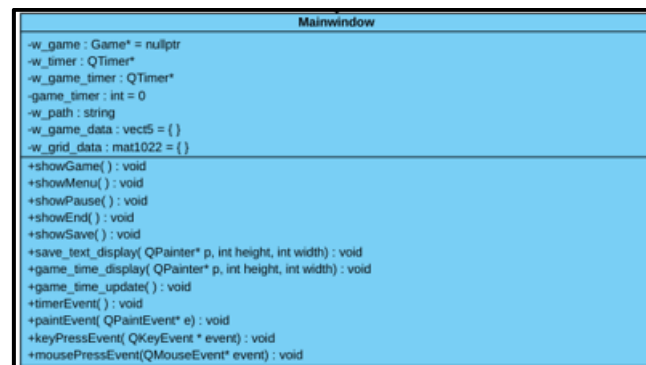


Figure 8 : Classe Mainwindow

La classe Mainwindow (voir Figure 8), elle, nous permet de gérer tout ce qui concerne l'affichage et les méthodes et attributs propres à QtCreator nous permettant d'avoir une interface graphique plus propre pour notre jeu. On retrouve les attributs `w_timer` et `w_game_timer` qui vont nous permettre de gérer respectivement la descente des tétramino dans le puit et l'incrémentation du temps du chronomètre tout au long de la partie. De plus, les

attributs `w_game_data` et `w_grid_data`, eux, nous servent à gérer une nouvelle fonctionnalité que nous avons ajouté : la sauvegarde de partie.

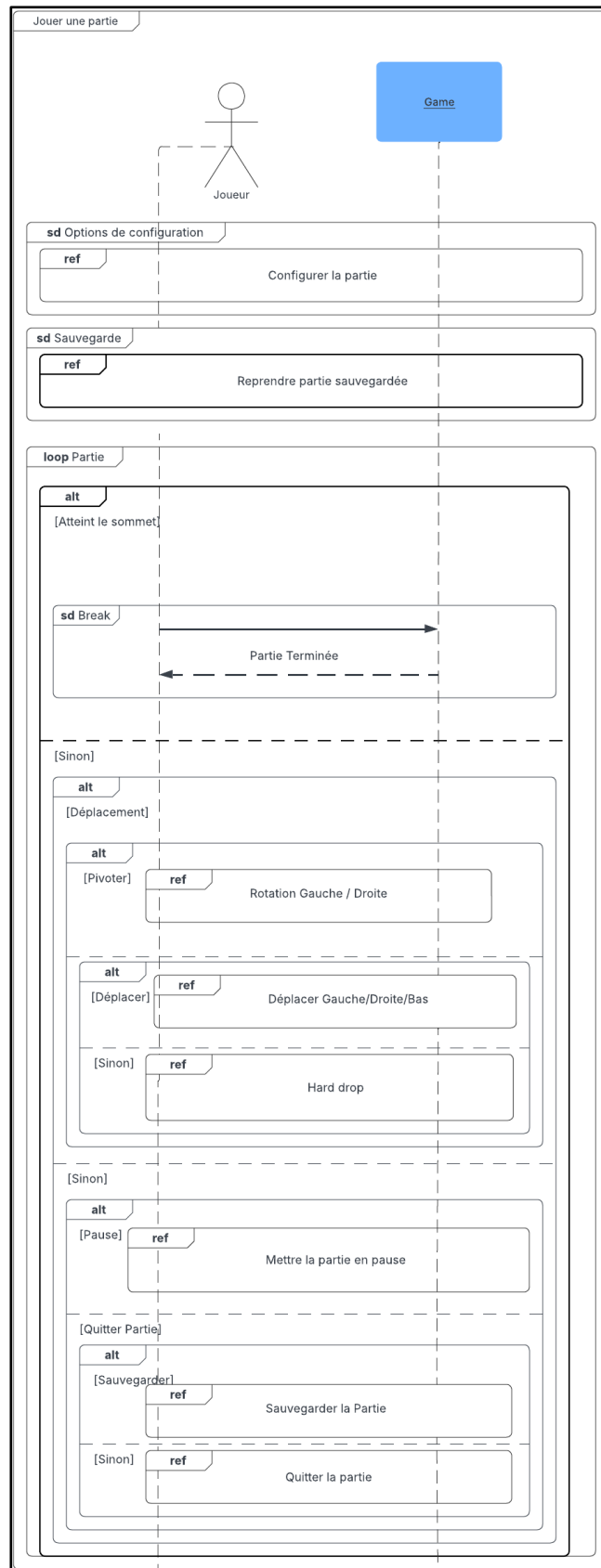


Figure 9 : Diagramme de séquences du jeu entier

Le diagramme de séquences ci-dessus (voir Figure 9) illustre le déroulement des interactions entre le joueur et la partie lorsqu'il arrive sur notre jeu. Dans ce diagramme nous retrouvons l'ensemble des interactions possibles entre ces deux entités. Nous allons donc maintenant détailler chacune des parties de celui-ci :

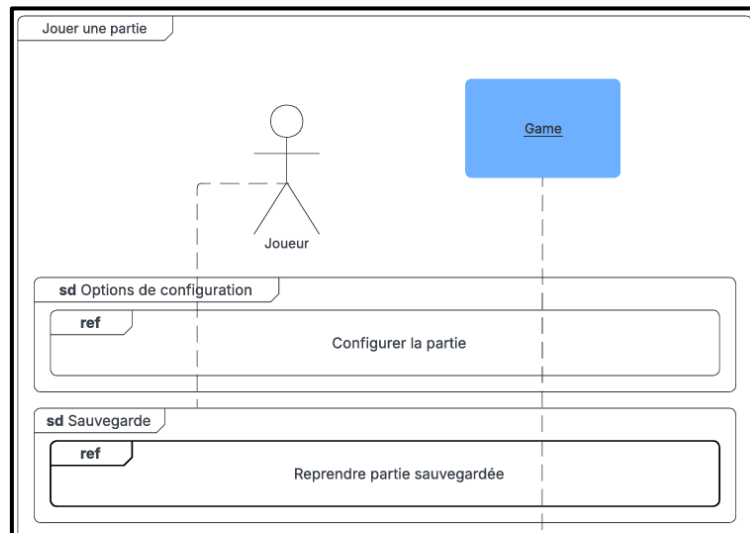


Figure 10 : Partie du diagramme de séquences qui ne concerne pas la partie

Cette partie de notre diagramme de séquences général (voir Figure 10) illustre la situation dans laquelle le joueur souhaite interagir avec le jeu mais ne souhaite pas encore lancer une nouvelle partie. Dans ce cas-là, il a deux options possibles ; choisir de configurer la partie et/ou consulter le menu ou alors reprendre une précédente partie ayant été sauvegardée au préalable.

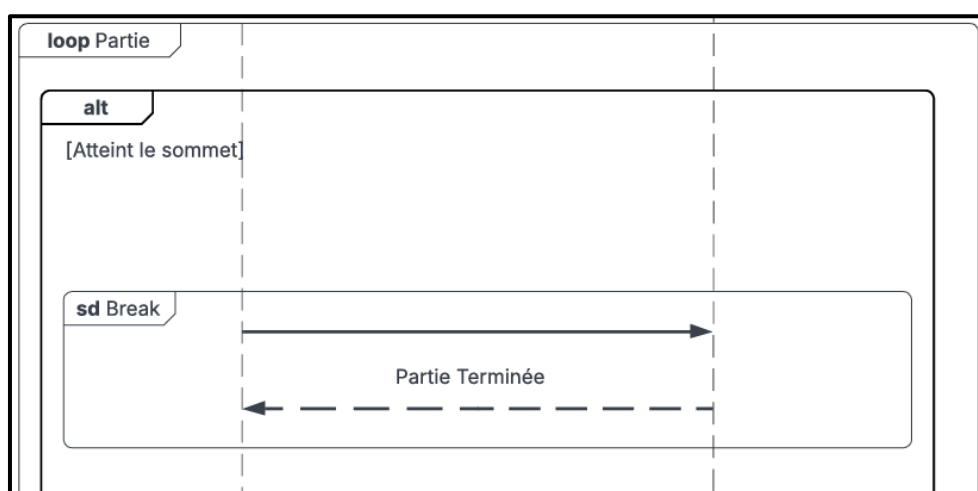


Figure 11 : Cas où la partie se termine dans la boucle partie

Cette partie-là du diagramme de séquences général (voir Figure 11), illustre le fait que lorsque les blocs atteignent le sommet, le joueur ne peut plus agir ; c'est la fin de la partie.

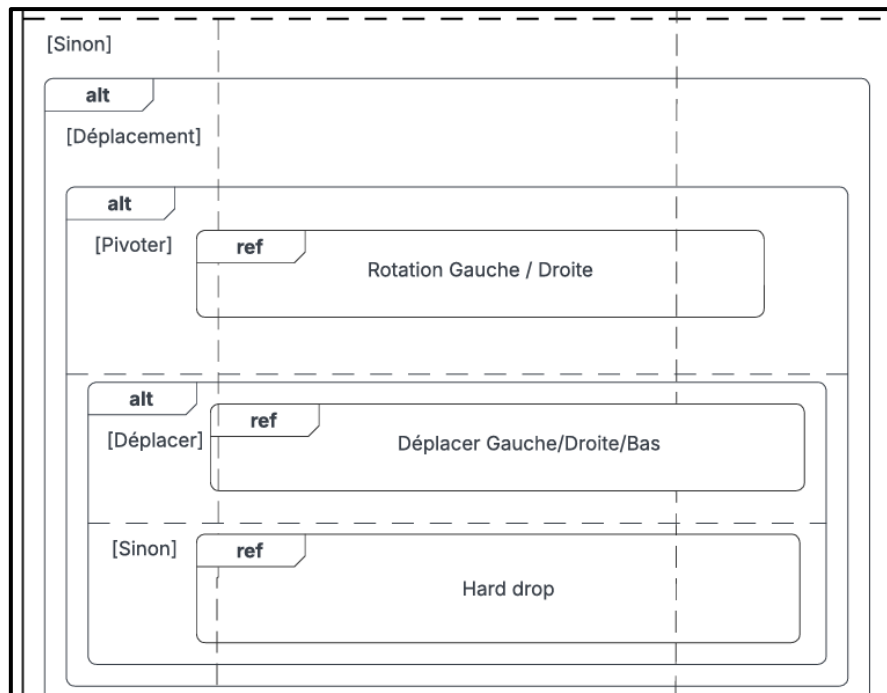


Figure 12 : Différents déplacements dans la boucle partie

Dans le cas où la partie n'est pas terminée, le joueur peut interagir de différentes façons avec la partie. Ici (voir Figure 12), la première grande catégorie d'interactions disponibles est détaillée dans un fragment combiné alternative : le déplacement. Deux façons de se déplacer se détachent, la rotation et le déplacement vers la gauche, la droite et le bas (voir Figures 14 et 15).

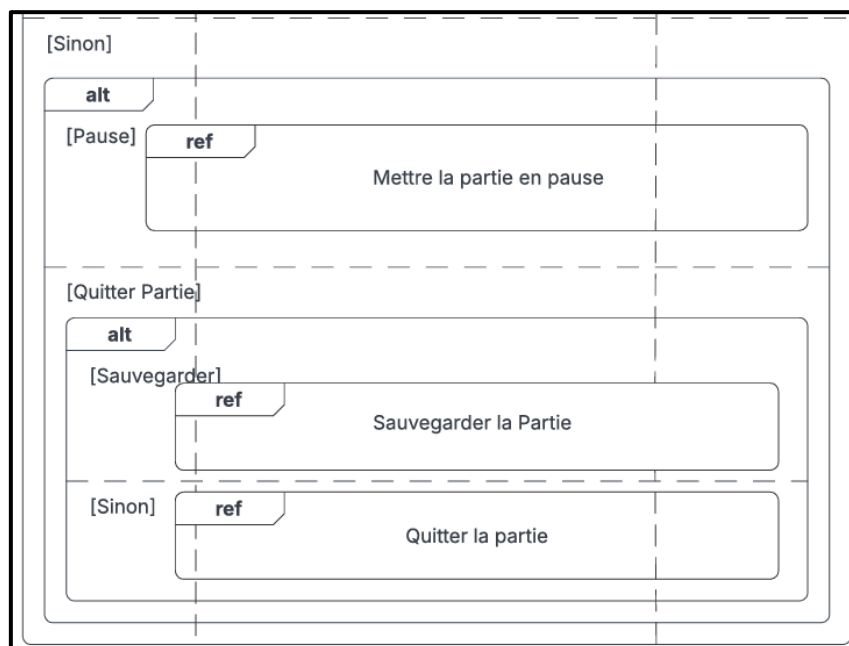


Figure 13 : Différentes façon de stopper la partie dans la boucle

Dans cette partie du fragment combiné alternative (voir Figure 13), nous nous retrouvons dans le cas de figure où le joueur ne souhaite pas interagir avec le jeu dans le but de déplacer le tétramino qui chute dans le puits. Plusieurs options s’offrent donc à lui : mettre en pause la partie ou quitter la partie sans ou après avoir effectué une sauvegarde de celle-ci.

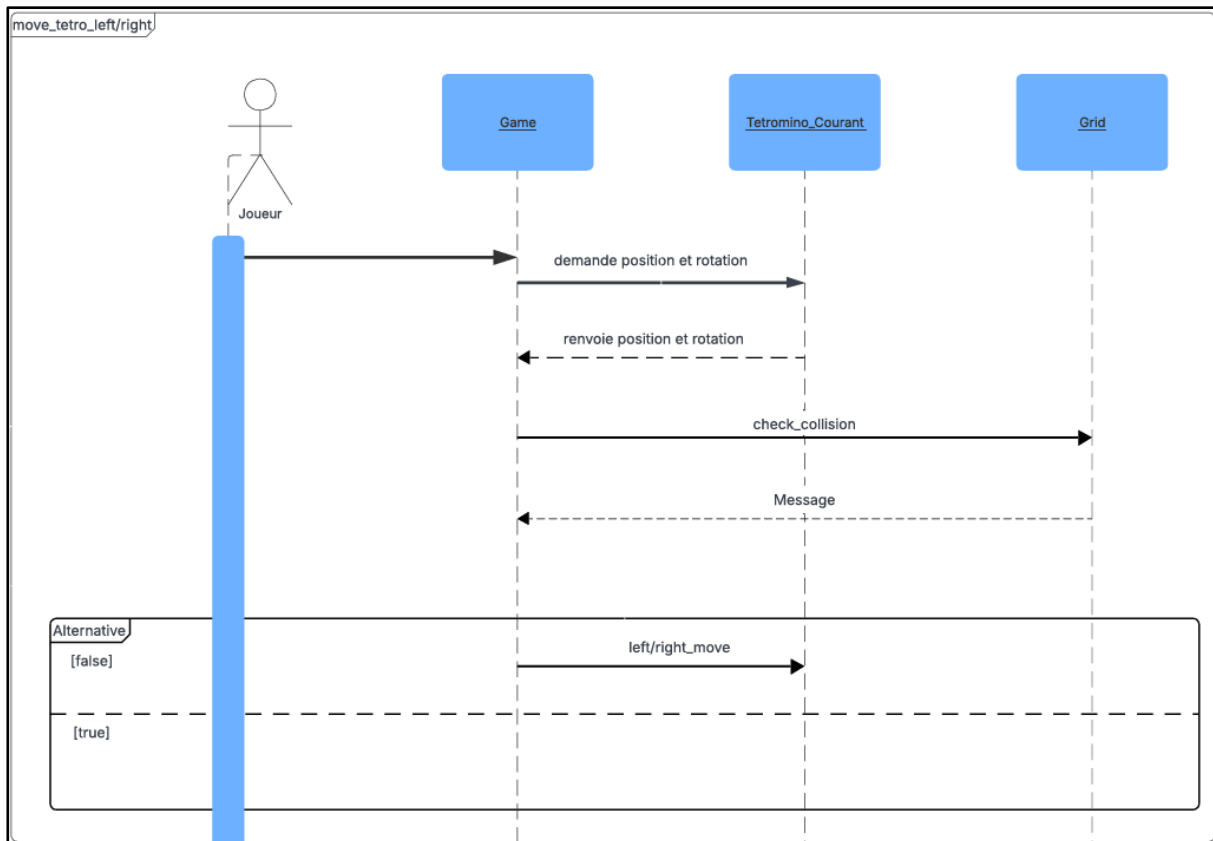


Figure 14 : Diagramme de séquences du déplacement gauche/droite

Dans le cas où le joueur décide de déplacer le bloc à gauche ou à droite (voir Figure 14), il interagit d’abord avec le jeu pour faire sa demande (en appuyant sur les touches flèche du clavier). Puis, le jeu récupère la position et la rotation du tétramino courant pour pouvoir ensuite vérifier si le déplacement du bloc causerait une collision. Ensuite, deux situations peuvent avoir lieu, soit le message indique qu’il y aurait une collision et le déplacement n’est pas réalisé, soit il est possible et effectué.

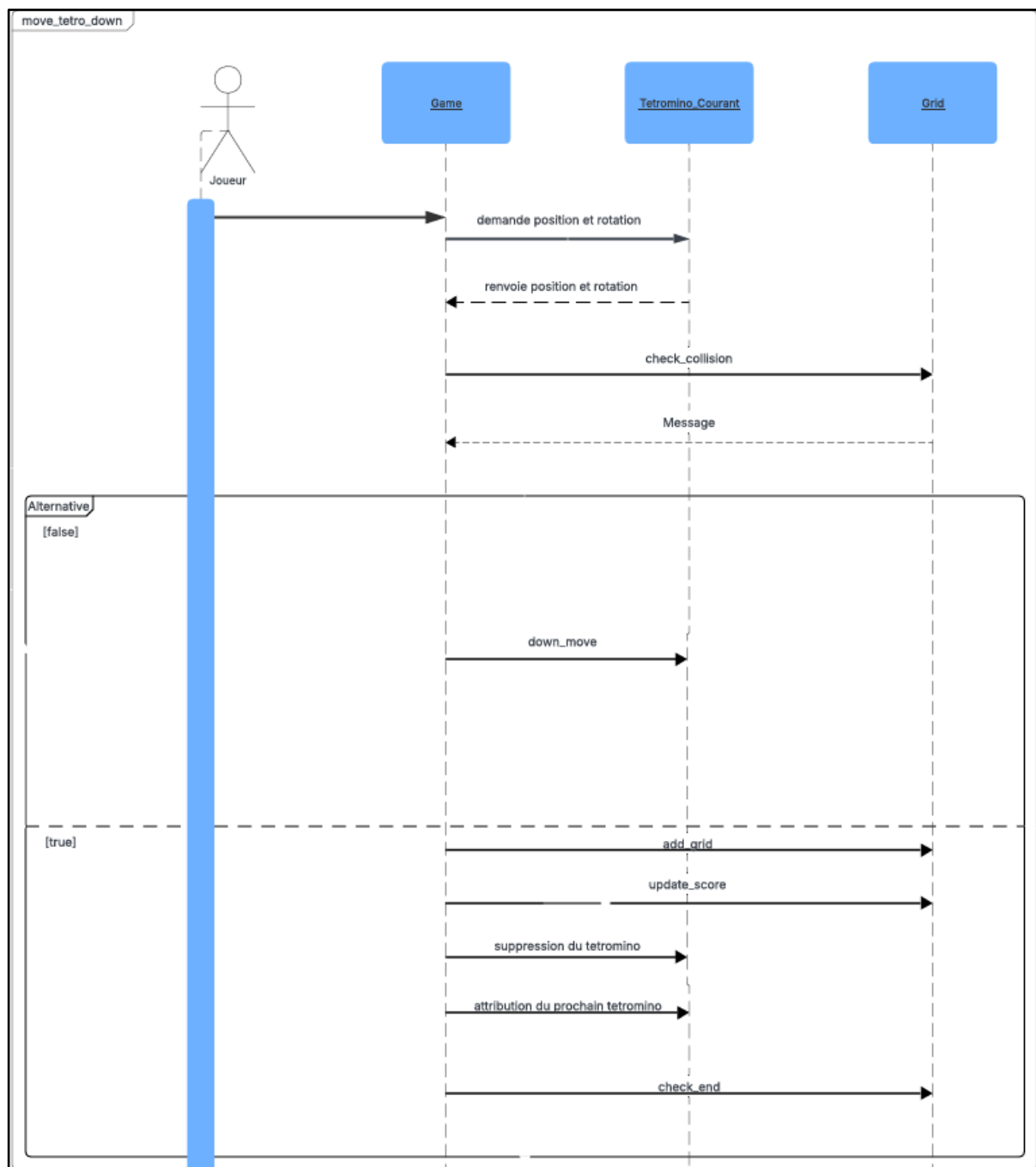


Figure 15 : Diagramme de séquences du déplacement vers le bas

Dans le cas d'un déplacement vers le bas (voir Figure 15), tout le début du processus est similaire à celui concernant le déplacement gauche/droite (voir Figure 14). En revanche le traitement de celui-ci lorsqu'il y a une collision n'est pas le même puisque lorsque le tétramino touche le bas de puit, il va automatiquement y être ajouté et le jeu va ensuite envoyer un nouveau tétramino en haut de puit tout en vérifiant si le joueur a perdu.

5.2 POINTS IMPORTANTS DU CODE

Certaines méthodes présentes au sein de notre code sont fondamentales pour le bon fonctionnement de notre jeu.

Parmi elles, on retrouve tout d'abord la méthode `check_collision` (voir Figure 16) appartenant à la classe `Game` (voir Figure 5) qui, comme son nom l'indique, permet de gérer plus facilement les collisions des tétramino avec le puit. Cette méthode renvoie vrai si elle détecte une collision et faux dans le cas contraire.

Elle prend en paramètres un tableau 4×4 qui contient la forme du tétramino dans son état de rotation actuel, ainsi que la position `x_pos` et `y_pos` de son coin supérieur gauche. En passe donc en paramètre l'éventuelle future position du bloc et on vérifie si celle-ci est possible.

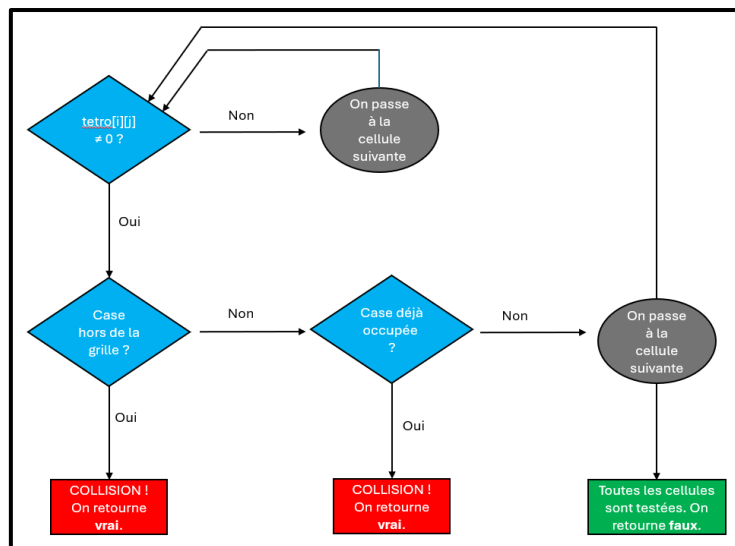


Figure 16 : Schématisation du fonctionnement de la méthode `check_collision`

La méthode `check_full_line` (voir Figure 17) de la classe « `Grid` » est elle aussi majeure puisqu'elle a plusieurs utilités très importantes pour notre code comme la détection des lignes complètes, la suppression des lignes complètes et les lignes situées au-dessus ainsi que l'envoi du nombre de lignes supprimées (nombre entier entre 0 et 4) ce qui permettra la mise à jour du score. Elle parcourt d'abord toutes les lignes de la grille en commençant par le bas et pour chacune d'entre-elles, une seconde boucle parcourt toutes leurs cases à la recherche d'une case vide. Si elle n'en trouve pas, elle en déduit que la ligne est pleine. Dans ce cas, une autre boucle parcourt toutes les lignes situées au-dessus de la ligne détectée pleine et en copie le contenu dans la ligne immédiatement inférieure. Enfin, l'opération est répétée pour chaque ligne pleine détectée et l'attribut `full_row` de la classe « `Game` » est mis à jour.

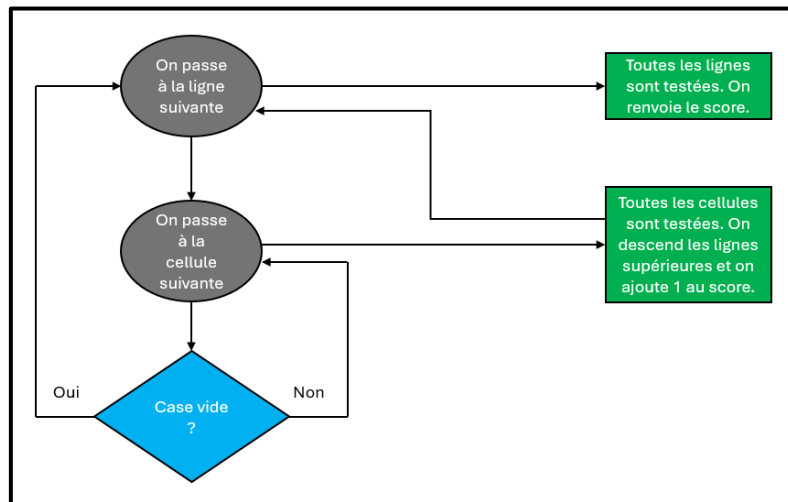


Figure 17 : Schématisation du fonctionnement de la méthode `check_full_line`

Enfin, les méthodes `display_projection` et `save` de la classe « Game » sont également très importantes puisque c'est en grande partie grâce à elles que nous avons réussi à implanter des fonctionnalités supplémentaires à notre jeu. La première prend en paramètre une copie du tétramino courant qu'elle affiche là où elle atterrirait dans le puit si le joueur faisait un `hard_drop` et elle l'affiche en gris. La seconde permet d'enregistrer la partie en cours sous la forme d'un fichier texte en prenant en paramètre le chemin vers le dossier de sauvegarde. Elle est alors associée à la méthode `on_continue_button_clicked` qui se charge de récupérer et de lire le fichier texte pour restaurer la partie dans son état d'origine. On ne pourra sauvegarder qu'une seule partie.

6 PRESENTATION DU JEU OBTENU

Finalement, nous obtenons un jeu qui, dès le lancement, affiche le menu principal sur l'écran du joueur (voir Figure 18). Celui-ci offre trois options différentes au joueur. Tout d'abord, il y a l'option « Play » qui va permettre de commencer une nouvelle partie de Tetris. Puis, il y a l'option « Continue » qui va permettre au joueur d'avoir accès à une partie qu'il avait mis en pause et surtout sauvegardée. Par ailleurs, il n'existe qu'un seul emplacement de sauvegarde sur notre jeu.

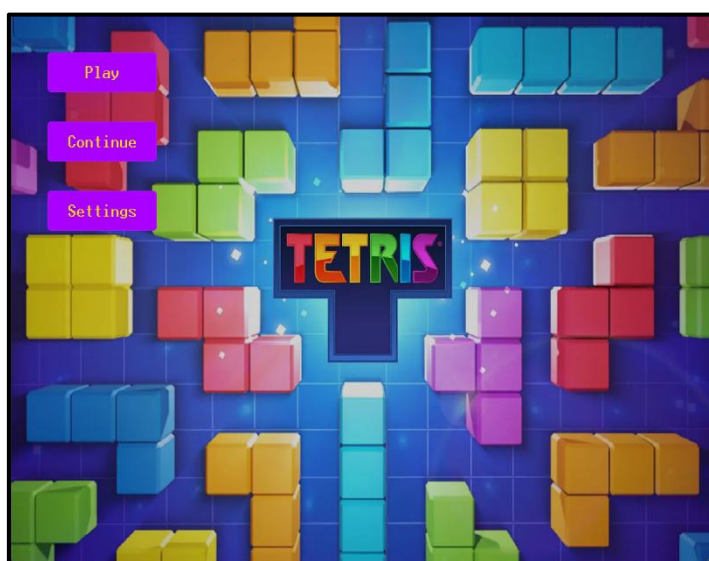


Figure 18 : Menu Principal du jeu

Une fois que le joueur a choisi l'option « Play », il a accès à un autre menu (voir Figure 19) qui va cette fois-ci lui permettre de choisir le niveau de difficulté. A nouveau, un choix entre 3 options différentes s'offre à lui. « Easy » est l'option la plus facile puisque l'ombre des pièces affichée et la vitesse de descente dans le puit augmente de 1/10e de seconde par niveau jusqu'au niveau 9. L'option « Medium » quant à elle, est déjà plus complexe. En effet, l'ombre des pièces n'est plus affichée. Enfin, l'option « Hard » est la plus difficile de toutes : il n'y a pas d'ombre et la vitesse de départ est doublée (50% du temps de base). En revanche, on perd 1/10e de seconde par niveau jusqu'au niveau 4.



Figure 19 : *Choix de la difficulté*

Le joueur a ensuite accès à l'écran de jeu (voir Figure 20). On y retrouve à droite le puit et à gauche un aperçu du tétramino suivant, le chronomètre le score et le niveau ainsi qu'un autre menu. Celui-ci permet au joueur d'avoir accès à un autre menu mettant en pause la partie (voir Figure 19) (bouton « Pause ») soit de quitter la partie (bouton « Quit »).

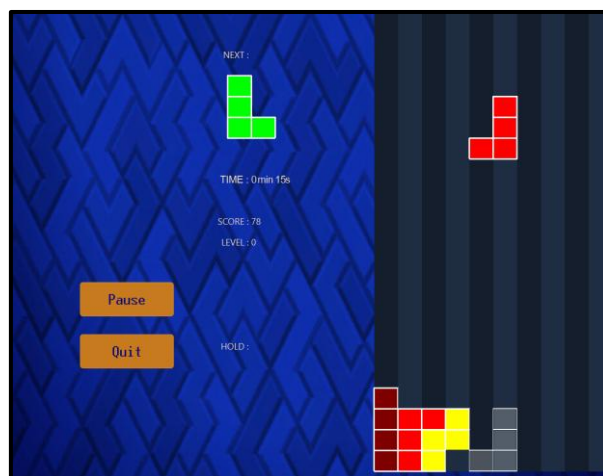


Figure 20 : *Configuration lors d'une partie de Tetris*

Ci-dessous (voir Figure 21), on retrouve le menu mentionné dans le paragraphe précédent. Il s'affiche notamment lorsque le joueur met la partie en pause et permet donc au joueur de choisir entre reprendre la partie (bouton « Get back ») et retourner dans le menu (bouton « Back to menu »).



Figure 21 : Pause en cours de jeu

Enfin, lorsque le joueur perd et que la partie se termine, un dernier menu s'affiche (voir Figure 22) permettant ainsi au joueur de commencer une nouvelle partie en choisissant l'option « Restart » ou de retourner au menu principal grâce à l'option « Back to menu ».

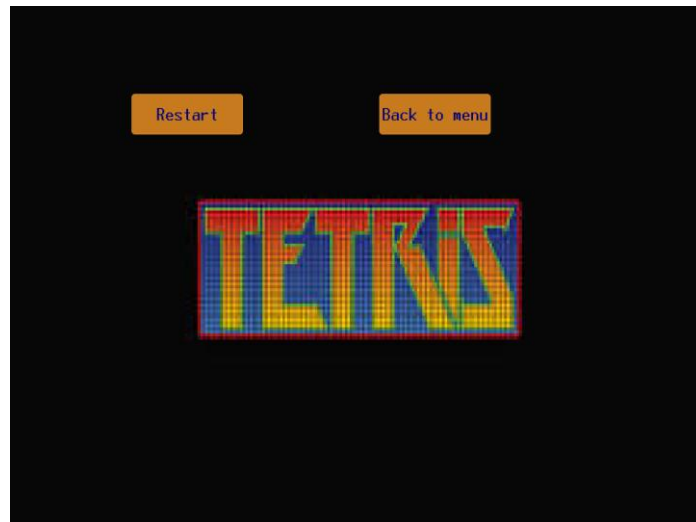


Figure 22 : Affichage lors d'une défaite

7 BILAN PERSONNEL

La réalisation de ce projet nous a permis de gagner en compétences sur le plan technique ainsi que sur le plan organisationnel.

En effet, nous avons pu mettre en pratique différentes notions abordées en cours. Sur le plan technique notamment, nous avons eu l'occasion d'approfondir notre maîtrise du langage C++ ainsi que de la bibliothèque graphique Qt. Nous avons aussi pu mieux comprendre le fonctionnement de la programmation orientée objet, notamment lorsque nous avons défini les classes principales de notre modèle et mis en place les relations entre celles-ci.

Nous avons cependant rencontré plusieurs défis au cours du développement et notamment lors de l'implantation de certaines fonctionnalités indispensables du jeu telles que la gestion des collisions, la suppression des lignes complètes ou encore la mise à jour du score ont nécessité de nombreux ajustements. Grâce à ces difficultés, nous avons pu développer notre capacité d'analyse ainsi que notre aptitude à résoudre des problèmes complexes.

Enfin, travailler en groupe nous a permis d'apprendre à organiser la répartition des tâches, à coordonner notre travail et à communiquer afin d'assurer la cohérence globale de notre travail. Nous avons donc développé nos compétences en gestion de projet et en travail d'équipe.

CONCLUSION

Finalement, notre jeu propose bien toutes les fonctions essentielles du Tetris telles que : la chute, le déplacement et la rotation des pièces, la suppression des lignes, l’affichage du score et la possibilité d’avoir plusieurs niveaux de difficulté. De plus, nous avons pu ajouter à cela d’autres fonctionnalités telles que l’apparition de l’ombre des pièces dans le niveau le plus simple, la possibilité de sauvegarder une partie ou encore le fait de pouvoir « garder en mémoire » une pièce.

Enfin, nous pourrions envisager d’autres axes d’amélioration pour améliorer notre logiciel, tels que la mise en place d’un système d’enregistrement du meilleur score qui permettrait au joueur de conserver ses performances, ou encore l’ajout d’un système de thèmes visuels personnalisables.

BIBLIOGRAPHIE

- [1] « *Tetris* », *Wikipédia*. 7 novembre 2025. Consulté le: 30 janvier 2026. [En ligne]. Disponible sur: <https://fr.wikipedia.org/w/index.php?title=Tetris&oldid=230437009>
- [2] « Visual Paradigm - AI-Powered Visual Modeling ». Consulté le: 30 janvier 2026. [En ligne]. Disponible sur: <https://www.visual-paradigm.com/>
- [3] « Lucidchart | Diagrammes alimentés par l'intelligence », Lucidchart. Consulté le: 30 janvier 2026. [En ligne]. Disponible sur: <https://www.lucidchart.com/pages/fr>
- [4] « Download Source Package Offline Installers | Qt ». Consulté le: 30 janvier 2026. [En ligne]. Disponible sur: <https://www.qt.io/development/offline-installers>
- [5] « Convertir JPEG en JPG (En ligne et Gratuit) — Convertio ». Consulté le: 6 février 2026. [En ligne]. Disponible sur: <https://convertio.co/fr/jpeg-jpg/>
- [6] « Convertisseur : YouTube MP3, Tiktok MP4 et plus encore! - noTube ». Consulté le: 6 février 2026. [En ligne]. Disponible sur: <https://notube.lol/fr/youtube-app-311>

